@codedscul_05

DEVOPS HANDBOOK

— HANDWRITTEN HANDBOOK —

From Basics to Advanced.
Concepts. Tools. Real World Use Cases.
Interview Q&A. Best Practices.

PLAN → CODE → BUILD → TEST → RELEASE → DEPLOY → MONITOR

AUTOMATE EVERYTHING | DELIVER FASTER | IMPROVE QUALITY | SCALE SEAMLESSLY | COLLABORATE BETTER

WHAT'S INSIDE?

1 DevOps Fundamentals
- What is DevOps?
- Principles & Culture
- DevOps Lifecycle
- Benefits & Challenges

2 SDLC + DevOps Lifecycle
- SDLC Overview
- DevOps Lifecycle
- Shift Left Approach
- Continuous Everything

3 Version Control + Git + GitHub
- Git Basics
- Branching & Merging
- Git Workflow
- GitHub Essentials

4 CI/CD Fundamentals
- CI vs CD vs CD
- Key Concepts
- Benefits
- Use Cases

5 Jenkins + Build Automation
- Jenkins Basics
- Jobs & Pipelines
- Plugins
- Best Practices

6 Docker + Containerization
- Docker Basics
- Images & Containers
- Dockerfile
- Docker Compose

7 Kubernetes + Container Orchestration
- K8s Architecture
- Pods, Deployments
- Services & Ingress
- Scaling & Rollouts

8 Infrastructure as Code + Terraform
- IaC Concepts
- Terraform Basics
- Providers & Resources
- State & Modules

9 Configuration Management + Ansible
- Ansible Basics
- Inventory & Playbooks
- Modules & Roles
- Best Practices

10 Cloud + DevOps
- Cloud Models
- AWS / Azure / GCP
- IAM & Networking
- DevOps on Cloud

11 CI/CD Pipeline Design
- Pipeline Stages
- Design Best Practices
- Artifacts & Caching
- Parallelism & Gates

12 Monitoring + Logging
- Monitoring Concepts
- Prometheus & Grafana
- ELK Stack
- Alerts & Dashboards

13 Security + DevSecOps
- DevSecOps Culture
- SAST, DAST, SCA
- Secrets Management
- Compliance & Policies

14 Deployment Strategies + Release Management
- Deployment Strategies
- Release Types
- Rollback & Rollout
- Blue/Green, Canary

15 DevOps Tools + Quick Revision Notes
- DevOps Toolchain
- Cheat Sheets
- Quick Revision
- Interview Notes

CONCEPTS MADE SIMPLE | PRACTICAL & HANDS-ON | INTERVIEW READY | REAL WORLD FOCUSED | BUILD BETTER DELIVER FASTER

LEARN DEEPLY. AUTOMATE DAILY. DELIVER VALUE. REPEAT.

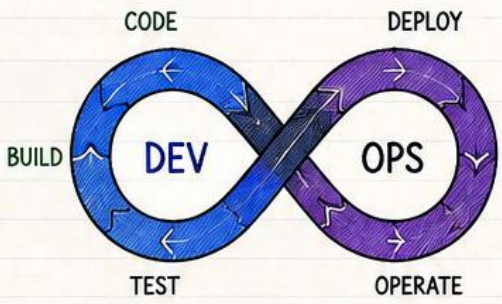
1. DEVOPS FUNDAMENTALS + CULTURE

BUILD BETTER. RELEASE FASTER. RUN STABLE.

WHAT IS DEVOPS?

DevOps is a combination of Development and Operations.

It is a culture, set of practices and tools that enables organizations to deliver applications and services at high velocity, with quality and reliability.



DEVOPS IS NOT...

- ✗ Just a tool
- ✗ Just Automation
- ✗ Just for Ops team
- ✗ Just about speed
- ✗ Replacing Dev or Ops

WHY DEVOPS?

- ✓ Faster Delivery
- ✓ Improved Collaboration
- ✓ Better Quality
- ✓ High Reliability
- ✓ Customer Satisfaction
- ✓ Business Value

KEY BENEFITS

- Continuous Delivery
- Reduced Time to Market
- Lower Failure Rate
- Scalability & Flexibility
- Better Communication
- Continuous Improvement

DEVOPS PRINCIPLES

- ✓ Customer-Centric
- ✓ End-to-End Ownership
- ✓ Collaborate & Communicate
- ✓ Automate Everything
- ✓ Measure & Monitor
- ✓ Learn & Improve Continuously
- ✓ Keep it Simple

DEVOPS CULTURE – THE FOUNDATION

DevOps is not just about tools or automation, it's about CULTURE.

COLLABORATION
Break down silos between Dev, Ops, QA, Security and Business teams.

COMMUNICATION
Open, transparent and frequent communication builds trust and alignment.

SHARED OWNERSHIP
Everyone is responsible for the entire lifecycle – from build to production.

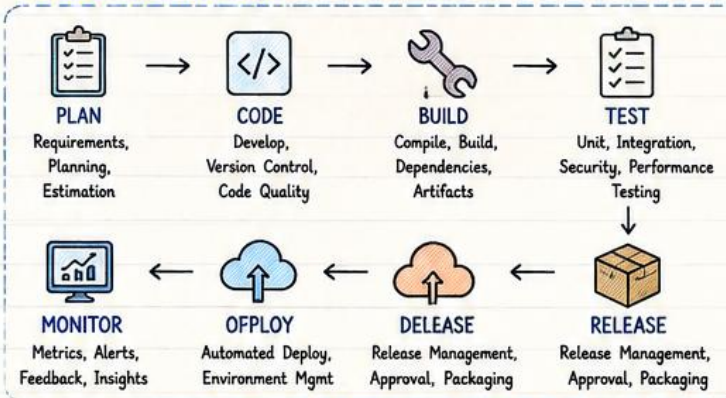
CONTINUOUS IMPROVEMENT
Inspect, adapt and improve processes, tools and skills continuously.

TRUST & RESPECT
Empower teams, value contributions and respect different perspectives.

TRADITIONAL IT vs DEVOPS

TRADITIONAL IT	VS	DEVOPS
Siloed Teams		Collaborative Teams
Waterfall Approach		Agile + Continuous Flow
Manual Processes		Automation Driven
Long Release Cycle		Frequent Releases
Focus on Tools		Focus on Culture + Outcomes
Blame on Failure		Learn from Failure
Change is Risk		Change is Normal

DEVOPS LIFECYCLE



DEVOPS PILLARS

PEOPLE
Right culture, collaboration, communication, shared goals.

PROCESS
Lean, Agile, repeatable, measurable processes.

TOOLS
Automation, integration, monitoring, orchestration.

PLATFORM
Cloud, Containers, Infrastructure, enabling scalability and speed.

> DevOps is a journey, not a destination. It's about people, process and technology working together to deliver value continuously.

DEVOPS = CULTURE + AUTOMATION + MEASUREMENT + SHARING

Work together. Automate everything. Deliver value.

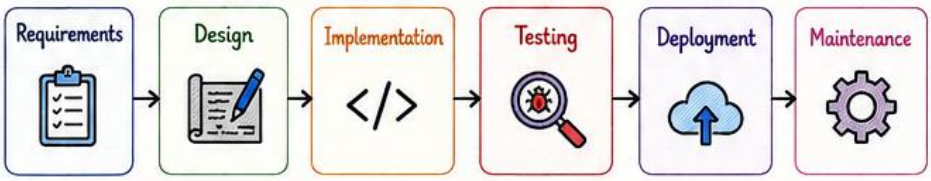
2. SDLC + DEVOPS LIFECYCLE

From Waterfall to Continuous Value Delivery

WHAT IS SDLC?

SDLC (Software Development Life Cycle) is a structured process used by traditional teams to build and deliver software. It follows a linear and phase-based approach.

TRADITIONAL SDLC (WATERFALL MODEL)



LIMITATIONS OF WATERFALL

- ✗ Late testing leads to more bugs
- ✗ Hard to accommodate changes
- ✗ Long release cycle
- ✗ High risk and cost of failure
- ✗ Poor collaboration and feedback

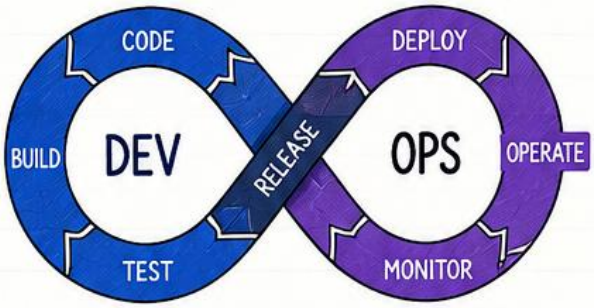
WHEN TO USE?

- ✓ When requirements are very clear and stable
- ✓ Small projects with low uncertainty
- ✓ Compliance / regulatory projects
- ✓ Teams with limited DevOps maturity

WHAT IS DEVOPS LIFECYCLE?

DevOps Lifecycle is a continuous, iterative approach that integrates Development (Dev) and Operations (Ops) to deliver value faster, with high quality and continuous improvement.

DEVOPS LIFECYCLE (INFINITY LOOP)



- CODE – Write & manage code
- BUILD – Compile, build artifacts
- TEST – Test early, test often
- RELEASE – Package & release
- DEPLOY – Deploy to environments
- OPERATE – Run & manage apps
- MONITOR – Monitor & improve

KEY BENEFITS OF DEVOPS LIFECYCLE

Faster Delivery
Ship features quickly and frequently.

High Quality
Continuous testing reduces defects and improves reliability.

Better Collaboration
Dev + Ops work together with shared goals & responsibility.

Continuous Improvement
Feedback loop helps to learn, optimize and improve continuously.

Business Value
Quick adaptation to changes and customer needs.

Stability & Reliability
Proactive monitoring ensures stable, secure and scalable applications.

SDLC vs DEVOPS LIFECYCLE

Aspect	Traditional SDLC (Waterfall)	DevOps Lifecycle
Approach	Linear, Phase-based	Continuous, Iterative
Focus	Project Completion	Continuous Value Delivery
Collaboration	Siloed Teams	Cross-functional Teams
Feedback	Late Feedback	Early & Continuous Feedback
Testing	After Development	Continuous Testing
Release	Infrequent, Big Releases	Frequent, Small Releases
Change Handling	Difficult & Costly	Easy & Embraced
Risk	High	Low
Tools	Manual / Limited Automation	Automation Everywhere
Goal	Deliver the Product	Deliver Value Continuously

HOW DEVOPS TRANSFORMS SDLC?

FROM
- ✗ Plan-Driven
- ✗ Siloed Teams
- ✗ Manual Processes
- ✗ Slow Releases
- ✗ React to Problems

TO
- ✓ Value-Driven
- ✓ Collaborative Teams
- ✓ Automated Processes
- ✓ Fast, Frequent Releases
- ✓ Prevent Problems

REAL-WORLD IMPACT

- Shorter time to market
- Happier customers
- Lower failure rate
- Better resource utilization
- Competitive advantage

DevOps is not a destination, it's a journey of
People + Process + Tools + Culture
working together to deliver value continuously.

3. VERSION CONTROL + GIT + GITHUB

Track Changes. Collaborate Better. Build Together.

WHAT IS VERSION CONTROL?

Version Control (VC) is a system that records changes to files over time so that you can recall specific versions later.

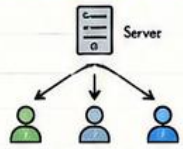
- ☑ Track changes
- ☑ Work in teams
- ☑ Revert to previous versions
- ☑ Maintain history
- ☑ Improve code quality & collaboration



History of Changes

TYPES OF VERSION CONTROL

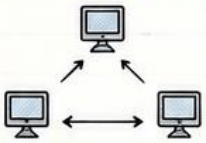
1. Centralized VCS (CVCS)



All versioned files are stored on a central server.

Example: SVN

2. Distributed VCS (DVCS)



Every developer has a full copy of the repository.

Example: Git (Most Popular)

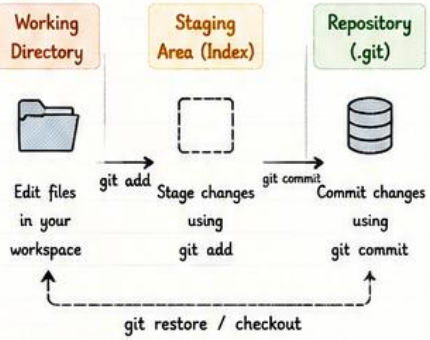
WHY GIT?

- ☑ Fast
- ☑ Distributed
- ☑ Branching & Merging
- ☑ Data Integrity
- ☑ Offline Work
- ☑ Powerful & Flexible

GIT BASICS

`git init`	Initialize a new Git repository
`git clone <url>`	Clone (copy) a repository
`git status`	Check the status of files
`git add <file>`	Stage changes to be committed
`git commit -m "msg"`	Commit staged changes
`git log`	Show commit history
`git diff`	Show changes between commits, files or working directory
`git restore <file>`	Discard changes in working directory
`git reset <file>`	Unstage a file
`git rm <file>`	Remove file from working directory and staging area

GIT WORKFLOW (3 AREAS)



FLOW: Modify Files (Working Directory) → `git add` (Staging Area) → `git commit` (Repository)

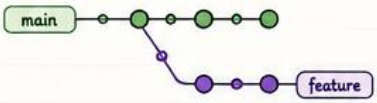
COMMON GIT COMMANDS

Command	Purpose
`git branch`	List branches
`git branch <name>`	Create new branch
`git checkout <branch>`	Switch branch
`git checkout -b <name>`	Create & switch branch
`git merge <branch>`	Merge branch into current branch
`git pull`	Fetch + Merge changes from remote
`git push`	Push changes to remote
`git stash`	Temporarily save changes
`git tag <v1.0>`	Create a tag
`git remote -v`	List remotes

GIT BRANCHING & MERGING

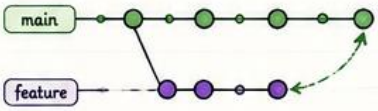
BRANCHING:

Creating a new line of development.



MERGING:

Integrating changes from one branch to another.



GITHUB BASICS

- GitHub is a web platform built on top of Git.
- It helps developers **host**, **manage** and **collaborate** on code.
- Key Features:
  - ✓ Repositories
  - ✓ Pull Requests
  - ✓ Issues
  - ✓ Projects
  - ✓ Actions (CI/CD)
  - 6 Wiki & Documentation



TYPICAL GITHUB FLOW

1. Create a repository on GitHub
2. Clone the repository locally
3. Make changes and commit
4. Push changes to GitHub
5. Create a Pull Request
6. Review & Merge

HOW GIT & GITHUB WORK TOGETHER



GIT CHEAT SHEET (QUICK REFERENCE)

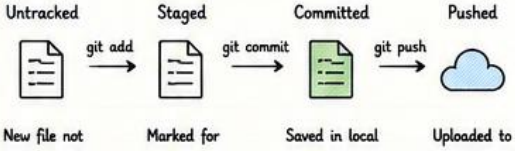
Initialize repo	`git init`
Clone repo	`git clone <url>`
Check status	`git status`
Add files	`git add .` (or `git add <file>`)
Commit	`git commit -m "message"`
View log	`git log --oneline --graph --all`
Create branch	`git branch <name>`
Switch branch	`git checkout <name>`
Create & switch	`git checkout -b <name>`
Merge branch	`git merge <name>`
Push to remote	`git push origin <branch>`
Pull from remote	`git pull origin <branch>`

BEST PRACTICES

- ✓ Commit small and meaningful changes
- ✓ Write clear commit messages
- ✓ Use branches for new features / fixes
- ✓ Pull latest changes before pushing
- ✓ Review code before merging
- ✓ Keep README.md updated
- ✓ Follow a consistent workflow



GIT FILE STATES



GOOD CODE IS IMPORTANT, BUT GREAT COLLABORATION BUILDS AMAZING SOFTWARE.

4. CI/CD FUNDAMENTALS

Automate. Integrate Early. Deliver Continuously.

WHAT IS CI/CD?

CI/CD stands for Continuous Integration, Continuous Delivery and Continuous Deployment.

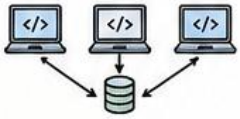
It is a practice that helps teams automate the process of integrating code, testing it and delivering it to users quickly and reliably.

Goal: Deliver value faster, with high quality and low risk.

CI vs CD

CONTINUOUS INTEGRATION (CI)

- Developers integrate code frequently (multiple times a day)
- Automated build & test
- Early bug detection
- Code always in a deployable state



CONTINUOUS DELIVERY/ DEPLOYMENT (CD)

- Automatically deliver code to the testing/production environment
- Ensures code is always ready to release
- Deployment can be manual (CD) or automatic (CD)



BENEFITS OF CI/CD

- ✓ Faster time to market
- ✓ Early detection of issues
- ✓ Improved code quality
- ✓ Consistent deployments
- ✓ Reduced manual errors
- ✓ Better collaboration
- ✓ High reliability & stability
- ✓ Easy rollbacks

CI/CD PIPELINE – THE FLOW

1. **PLAN** – Plan features, requirements and tasks.
2. **CODE** – Developers write code and push to repository.
3. **BUILD** – Compile code and resolve dependencies. Create build artifact.
4. **TEST** – Run automated tests (unit, integration, security, etc.).
5. **PACKAGE** – Package the build artifact for deployment.
6. **DEPLOY** – Deploy to staging / production environment.
7. **MONITOR** – Monitor applications, collect logs, get feedback.

Feedback Loop: Monitor → Learn → Improve

TYPES OF CI/CD

CONTINUOUS INTEGRATION (CI)

Automatically integrate code changes from multiple contributors into a shared repository multiple times a day.

CONTINUOUS DELIVERY (CD)

Automatically deliver code to a staging environment and keep it ready for release.

CONTINUOUS DEPLOYMENT (CD)

Automatically deploy every changed code to production without manual intervention.

KEY PRINCIPLES

Automate Everything
Automate build, test, code analysis, security, and deployment.

Small & Frequent Changes
Commit small changes often to reduce risk.

Early Testing
Test early and often to find bugs as soon as possible.

Fast Feedback
Get quick feedback from the system and users.

Continuous Improvement
Iterate and improve the process continuously.

CI/CD TOOLS (EXAMPLES)

Jenkins	Popular open-source automation server.
GitLab CI/CD	Built-in CI/CD with GitLab.
GitHub Actions	CI/CD service by GitHub.
CircleCI	Cloud-based CI/CD platform.
Azure DevOps	CI/CD from Microsoft.

CI/CD vs TRADITIONAL DEPLOYMENT

Aspect	Traditional Deployment	CI/CD
Integration	Manual, infrequent	Automatic, frequent
Testing	Manual testing	Automated testing
Deployment	Manual, time consuming	Automated, fast
Feedback	Late feedback	Early & continuous feedback
Risk	High	Low
Quality	Inconsistent	Consistent & reliable
Release Frequency	Weeks/Months	Daily / On-demand

BEST PRACTICES

- Keep the build fast and reliable.
- Automate tests and make them part of the pipeline.
- Maintain a single source repository.
- Use versioned artifacts and immutable deployments.
- Monitor everything and set up alerts.
- Secure the pipeline and manage secrets properly.
- Keep improving the pipeline and process.

CI/CD helps teams deliver high-quality software faster and more reliably through automation, collaboration and continuous improvement.

5. JENKINS + BUILD AUTOMATION



Automate Builds. Run Tests. Deliver Faster.

WHAT IS JENKINS?

Jenkins is an open-source automation server used to automate the building, testing, and deployment of software.

- ✓ Helps implement CI/CD pipelines
- ✓ Extensible with Plugins
- ✓ Easy to install and configure
- ✓ Supports distributed builds



BUILD AUTOMATION

Build automation is the process of automatically converting source code into an executable application.

It includes:

- ✓ Compiling the code
- ✓ Running tests
- ✓ Packaging the application
- ✓ Generating reports
- ✓ Deploying the artifact

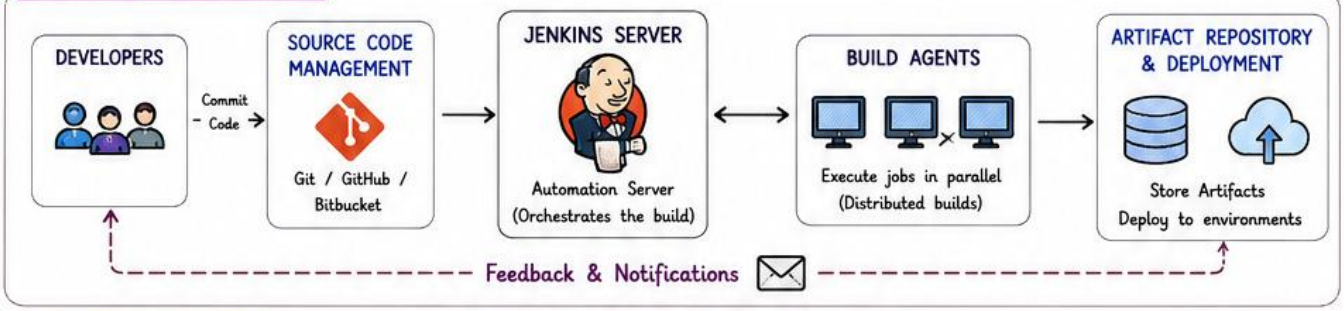


KEY BENEFITS

- ✓ Automates repetitive tasks
- ✓ Ensures consistency
- ✓ Early detection of errors
- ✓ Faster feedback to developers
- ✓ Integration of code frequently
- ✓ Improved productivity
- ✓ Better software quality



JENKINS ARCHITECTURE



JENKINS JOB TYPES

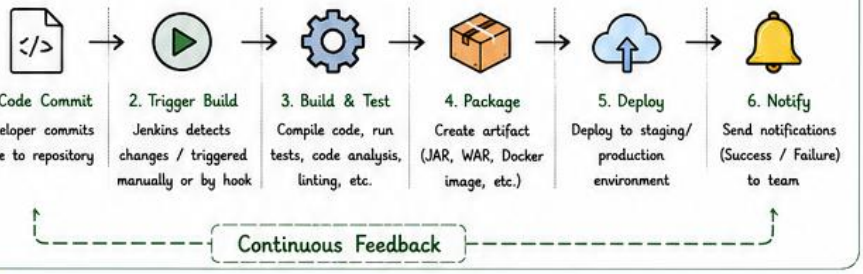
Freestyle Project
Basic job with configurable build steps.

Pipeline
Define the entire CI/CD pipeline as code.

Multi-configuration Project
For projects that need multiple configurations.

Folder
Group related jobs and manage them.

BUILD AUTOMATION WORKFLOW (TYPICAL)



JENKINS PIPELINE EXAMPLE (Declarative)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build') {
      steps { sh 'mvn clean package' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Package') {
      steps { archiveArtifacts artifacts: 'target/*.jar' }
    }
    stage('Deploy') {
      steps { sh 'echo "Deploying..."' }
    }
  }
  post {
    always {
      echo 'Build Completed'
    }
  }
}
```

Stages Explained

- Checkout – Get code from repo
- Build – Compile the code
- Test – Run unit/integration tests
- Package – Create artifact
- Deploy – Deploy to env
- Post – Final actions/notifications



POPULAR PLUGINS

Pipeline	Pipeline	Create and run CI/CD pipelines
Git	Git	Integrate with Git repositories
JUnit	JUnit	Publish test results
SonarQube Scanner	SonarQube Scanner	Code quality analysis
Email Extension	Email Extension	Send email notifications
Docker	Docker	Build and manage Docker images
Slack Notification	Slack Notification	Send Slack alerts

BEST PRACTICES

- ✓ Keep jobs independent and modular
- ✓ Use version control for pipeline scripts (Jenkinsfile)
- ✓ Use agents and parallel builds for faster execution
- ✓ Clean workspace before build
- ✓ Use meaningful job and stage names
- ✓ Notify team on build failure immediately
- ✓ Secure Jenkins with authentication & authorization



USE CASES

- Automate build & test for every commit
- Continuous integration of microservices
- Automated deployments to multiple environments
- Nightly builds and scheduled jobs
- Integration with dev tools (Git, Docker, Kubernetes)

JENKINS vs MANUAL PROCESS

Aspect	Manual	With Jenkins
Build Time	High	Low
Errors	More	Less
Consistency	Low	High
Feedback	Slow	Fast
Repeatability	Difficult	Easy
Scalability	Poor	Excellent

AUTOMATE TODAY. DELIVER BETTER TOMORROW.



6. DOCKER + CONTAINERIZATION

Build Once. Run Anywhere.

WHAT IS DOCKER?

Docker is an open-source platform that uses containerization to package an application and all its dependencies together so that it can run consistently on any environment.

Container = Lightweight, Portable, Consistent

HOW DOES DOCKER WORK?

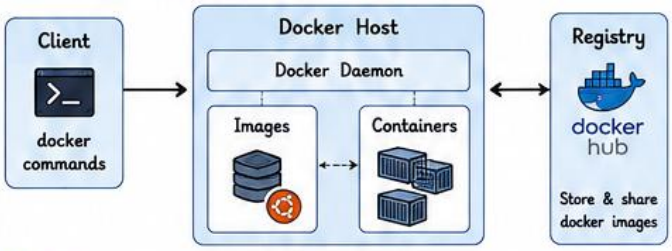
Write Code → Build Image → Run Container → Share & Deploy

Docker packages the app and dependencies into an image. The image runs as a container.

BENEFITS

- Lightweight & Fast
- Consistent Environments
- Portability
- Better Resource Utilization
- Isolation & Security
- Easy Scaling & Deployment
- Developer Productivity

DOCKER ARCHITECTURE



- Client: Sends commands to the Docker daemon.
- Docker Host: Builds, runs and manages containers.
- Registry: A storage for Docker images (e.g., Docker Hub).

DOCKERFILE – BLUEPRINT FOR IMAGE

```
# Example Dockerfile
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

FROM	:	Base image
WORKDIR	:	Set working directory
COPY	:	Copy files
RUN	:	Execute commands
EXPOSE	:	Expose port
CMD	:	Default command

Build Image:

```
docker build -t myapp:1.0 .
```

COMMON DOCKER COMMANDS

Command	Description
`docker build -t <image> .`	Build image from Dockerfile
`docker images`	List all images
`docker run -d -p 8080:80 <image>`	Run container in detached mode
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <container_id>`	Stop a running container
`docker rm <container_id>`	Remove a container
`docker rmi <image>`	Remove an image
`docker logs <container_id>`	View container logs
`docker exec -it <container_id> bash`	Execute command in container

CONTAINER LIFECYCLE (WORKFLOW)



Pull Image	Run Container	Use / Access	Stop Container	Remove Container
`docker pull nginx`	`docker run nginx`	Access app in browser	`docker stop container_id`	`docker rm container_id`

IMAGES vs CONTAINERS

IMAGE
- Read-only template
- Built from Dockerfile
- Stored in registry
- Cannot be changed

VS

CONTAINER
- Runnable instance of image
- Read-Write layer on top
- Ephemeral (can be removed)
- Isolated environment

VOLUMES (PERSIST DATA)



- Data in container is ephemeral.
- Use volumes to persist data even if container is removed.

Example:

```
docker run -v mydata:/data nginx
```

NETWORKING (CONNECT CONTAINERS)



- Docker provides networking between containers.
- Default bridge network, custom networks.

Example:

```
docker network create mynet
docker run --network mynet nginx
```

DOCKER COMPOSE (MULTI-CONTAINER APPS)

Define and run multi-container applications using a YAML file.

File: docker-compose.yml

```
version: '3.8'
services:
  web:
    image: nginx
    ports:
      - "80:80"
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
```

Commands:

```
docker compose up -d
docker compose ps
docker compose down
```

BEST PRACTICES

- ✓ Use official and minimal base images (Alpine).
- ✓ Keep images small (multi-stage builds).
- ✓ Do not run as root user.
- ✓ Use .dockerignore to exclude unnecessary files.
- ✓ Scan images for vulnerabilities.
- ✓ Tag images properly.
- ✓ Remove unused images and containers.

USE CASES

Microservices	CI/CD Pipelines	Legacy Applications	Cloud & Hybrid	Development
Run each service in its own container	Consistent build & test environments	Containerize old apps without changing code	Deploy anywhere – On-prem or Cloud	Same environment for all developers

Docker makes applications portable, consistent and easy to deploy across any environment!

7. KUBERNETES + CONTAINER ORCHESTRATION

Automate Deployment. Scale Seamlessly. Run Anywhere.

WHAT IS KUBERNETES?

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, operation and management of containerized applications across a cluster of machines.

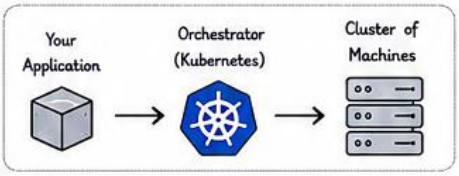
Kubernetes =
Automate, Scale,
Heal, Manage Containers

WHY KUBERNETES?

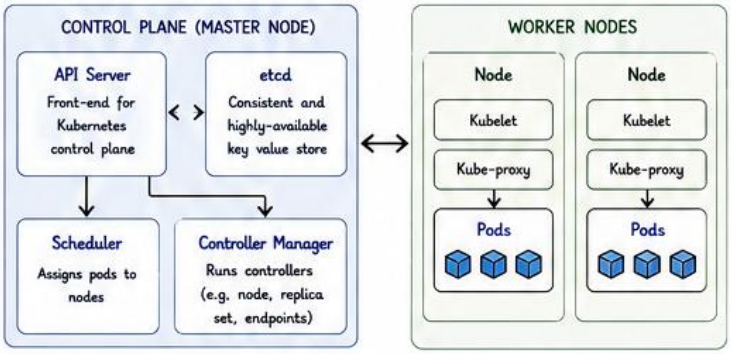
- Automates deployment and rollbacks
- Self-healing (restarts failed containers)
- Automatic scaling (up & down)
- Load balancing and service discovery
- Efficient resource utilization
- Consistent across environments
- Supports microservices architecture

CONTAINER ORCHESTRATION

It is the process of automating the deployment, management, scaling, networking and availability of containers.



KUBERNETES ARCHITECTURE

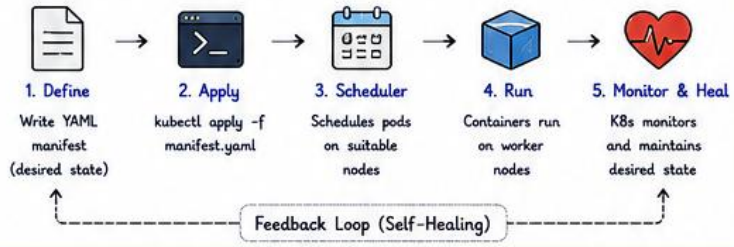


- **Control Plane:** Manages the cluster and makes global decisions.
- **Worker Nodes:** Run the actual containerized applications.

KEY KUBERNETES OBJECTS

Object	Purpose
Pod	Smallest deployable unit. One or more containers.
ReplicaSet	Ensures specified number of pod replicas are running.
Deployment	Manages ReplicaSets and provides declarative updates.
Service	Stable network endpoint to access pods.
Namespace	Virtual cluster within a cluster.
ConfigMap	Store configuration data in key-value pairs.
Secret	Store sensitive data (passwords, tokens, keys).
PersistentVolume (PV)	Piece of storage in the cluster.
PersistentVolumeClaim (PVC)	Request storage using PV.
Ingress	Manage external access (HTTP/HTTPS) to services.

KUBERNETES WORKFLOW



KUBECTL – BASIC COMMANDS

Command	Description
`kubectl get nodes`	List all nodes
`kubectl get pods`	List all pods
`kubectl get all`	List all resources
`kubectl dessribe <resource> <name>`	Show details of a resource
`kubectl logs <pod-name>`	View logs of a pod
`kubectl exec -it <pod> -- bash`	Open shell in a pod
`kubectl apply -f <file.yaml>`	Create/Update resources
`kubectl delete -f <file.yaml>`	Delete resources
`kubectl scale deployment <name> --replicas=3`	Scale deployment
`kubectl rollout status deployment/<name>`	Check rollout status

K8S DEPLOYMENT EXAMPLE

deployment.yaml (Example)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name:  myapp
  labels:
    app:  myapp
spec:
  replicas:  3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app:  myapp
    spec:
      containers:
      - name: myapp-container
        image: nginx:1.25
        ports:
          - containerPort:  80
```

Service (Expose)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
  - port:  80
    targetPort:  80
    nodePort:  30080
```

Access

`http://<NodeIP>:30080`

SERVICE TYPES

ClusterIP (Default)
Exposes service on an internal IP within the cluster.

NodePort
Exposes service on each node's IP at a static port.

LoadBalancer
Exposes service externally using cloud provider's load balancer.

ExternalName
Maps service to an external DNS name.

POD SCALING

Manual Scaling

```
kubectl scale deployment myapp --replicas=5
```

Auto Scaling (HPA)



Autoscaling Example (HPA YAML)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas:2
  maxReplicas:  10
  maxReplicas: 10
  metrics:
    type: Resource
      resource:
        name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

BEST PRACTICES

- Keep containers stateless.
- Use Deployments, not directly Pods.
- Set resource requests and limits.
- Use health checks (liveness & readiness probes).
- Store configs in ConfigMaps and secrets in Secrets.
- Use namespaces to separate environments.
- Monitor with Prometheus + Grafana.
- Keep images small and secure.

USE CASES

Microservices Applications
CI/CD Pipelines
Multi-Cloud & Hybrid Cloud
High Availability Applications
Batch Jobs & CronJobs
Machine Learning Workloads

KUBERNETES AUTOMATES CONTAINERS, YOU FOCUS ON BUILDING GREAT APPLICATIONS!

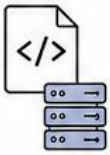
8. INFRASTRUCTURE AS CODE + TERRAFORM

Code Your Infrastructure. Provision Consistently. Scale Confidently.

WHAT IS IaC?

Infrastructure as Code (IaC) means managing and provisioning infrastructure through code instead of manual processes.

- Versionable
- Repeatable
- Consistent
- Automated
- Collaborative



IaC = Treat infrastructure like software.

WHY IaC?

- ✓ Eliminates manual errors
- ✓ Faster provisioning and changes
- ✓ Consistency across environments
- ✓ Easy rollback and recovery
- ✓ Better collaboration and review
- ✓ Cost-effective and scalable



WHAT IS TERRAFORM?

Terraform is an open-source IaC tool by HashiCorp. It allows you to define, provision and manage infrastructure across multiple cloud providers using a simple configuration language called HCL.

HashiCorp Terraform

HOW TERRAFORM WORKS?

1. **Write** — Write infrastructure as code (.tf files)
2. **Plan** — Terraform creates an execution plan
3. **Apply** — Terraform provisions or updates resources
4. **Manage** — Manage & track infrastructure
5. **Destroy** — Terraform destroys resources when needed

Plan → Review → Apply → Manage → Destroy

KEY FEATURES

- Declarative configuration language (HCL)
- Execution plan before any change
- Tracks state of infrastructure
- Manages dependencies automatically
- Supports multiple providers (AWS, Azure, GCP, etc.)
- Extensible with modules and providers
- Drift detection and self-healing

TERRAFORM CORE CONCEPTS

Concept	Description
Providers	Plugins that interact with APIs of platforms (AWS, Azure, etc.)
Resources	Infrastructure components (EC2, S3, VPC, etc.)
Data Sources	Fetch information about existing resources
Variables	Input values to make configuration dynamic
Outputs	Show important information after apply
State	JSON file that maps real resources to config
Modules	Reusable configuration for multiple setups

Terraform Flow

.tf Files
↓
terraform init
↓
terraform plan
↓
terraform apply
↓
Infrastructure

SUPPORTED PROVIDERS (EXAMPLES)

- **AWS** — Amazon Web Services
- **Azure** — Microsoft Azure
- **Google Cloud** — Google Cloud Platform
- **Kubernetes** — Kubernetes Cluster
- **Docker** — Docker and Docker Hub

... and many more (1000+ Providers)

EXAMPLE: CREATE AWS S3 BUCKET

main.tf

```
provider "aws" {
  region = "us-east-1"
}
resource "aws_s3_bucket" "mybucket" {
  bucket = "my-terraform-bucket-123"
  acl    = "private"
  tags = {
    Environment = "Dev"
    ManagedBy   = "Terraform"
  }
}
output "bucket_name" {
  value = aws_s3_bucket.mybucket.bucket
}
```

terraform.tfvars (variables)

```
aws_region = "us-east-1"
bucket_name = "my-terraform-bucket-123"
```

Expected Output

```
bucket_name = my-terraform-bucket-123
```

TERRAFORM STATE (state.tfstate)

```
{
  "version": 4,
  "resources": [
    {
      "type": "aws_s3_bucket",
      "name": "mybucket",
      "instances": [
        {
          "attributes": {
            "bucket": "my-terraform-bucket-123",
            "region": "us-east-1",
            "acl": "private"
          }
        }
      ]
    }
  ]
}
```

State File Stores:

- Mapping of config to real resources
- Resource metadata
- Dependencies
- Used for planning and change management

Note:

Never edit state file manually!

TERRAFORM COMMANDS CHEAT SHEET

Command	Purpose
terraform init	Initialize Terraform, download providers
terraform validate	Validate configuration files
terraform fmt	Format configuration files
terraform plan	Create an execution plan
terraform apply	Apply the changes and provision resources
terraform destroy	Destroy all resources defined
terraform show	Show current state or plan
terraform output	Show outputs from the state

BEST PRACTICES

- ✓ Use remote state backend (S3, Azure Blob, GCS)
- ✓ Enable state locking (DynamoDB, etc.)
- ✓ Use variables and locals for reusability
- ✓ Follow environment separation (dev, test, prod)
- ✓ Use modules for DRY and standardization
- ✓ Review plans before apply
- ✓ Store Terraform code in version control
- ✓ Use workspaces for multiple environments
- ✓ Apply least privilege for cloud credentials

USE CASES

- Provision Cloud Infrastructure (Compute, Network, Storage)
- Deploy Multi-tier Applications
- Create CI/CD and DevOps Tooling
- Disaster Recovery & High Availability
- Environment Consistency (Dev, Test, Prod)
- Cost Optimization and Governance

TERRAFORM + IaC = FASTER DEPLOYMENTS, FEWER ERRORS, BETTER INFRASTRUCTURE!

9. CONFIGURATION MANAGEMENT + ANSIBLE

Automate. Configure. Orchestrate. Repeatable Infrastructure.

WHAT IS CONFIGURATION MANAGEMENT?

Configuration Management is the process of automatically maintaining the desired state of systems, ensuring consistency across environments.

- Automate repetitive tasks
- Ensure consistency & compliance
- Reduce manual errors
- Speed up provisioning & changes
- Improve scalability & reliability

Goal: Keep systems in the desired state, all the time.

WHAT IS ANSIBLE?

Ansible is an open-source configuration management, application deployment and task automation tool.

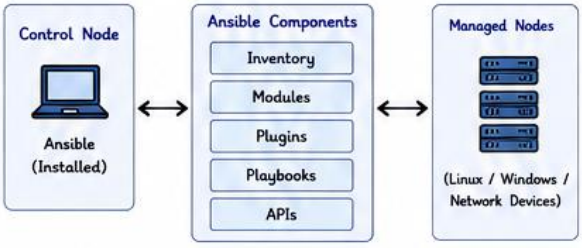
- Agentless (No software to install on managed nodes)
- Uses SSH (Linux) / WinRM (Windows)
- Simple YAML based language
- Powerful and easy to learn
- Idempotent (Safe to run multiple times)

SIMPLE IT AUTOMATION

WHY ANSIBLE?

- Agentless architecture
- Easy to install and use
- Human readable (YAML)
- Large community and modules
- Cross-platform (Linux, Windows, Network)
- Extensible
- Idempotent and reliable
- Perfect for DevOps & Cloud Automation

ANSIBLE ARCHITECTURE

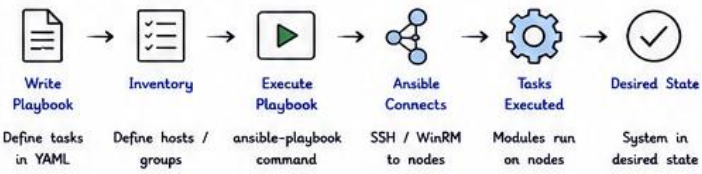


- **Control Node:** Where Ansible is installed and commands are executed.
- **Managed Nodes:** Nodes that are managed by Ansible.
- **Communication:** Over SSH (Linux), WinRM (Windows).
- **No Agents:** No software or daemon required on managed nodes.

USE CASES

- Server Provisioning
- Application Deployment
- Configuration Management
- Patch Management
- User Management
- Network Automation
- Cloud Infrastructure Management

HOW ANSIBLE WORKS?



Ansible is agentless, push-based and uses modules to perform tasks.

KEY ANSIBLE CONCEPTS

Concept	Description
Inventory	List of managed hosts and groups.
Modules	Reusable units of code to perform tasks.
Playbooks	YAML files containing plays, tasks and variables.
Roles	Organize playbooks and tasks in a structured way.
Variables	Store values and make playbooks dynamic.
Facts	Information collected from managed nodes.
Handlers	Tasks triggered only when notified by other tasks.
Idempotency	Running a task multiple times gives same result.

PLAYBOOK EXAMPLE (YAML)

```yaml
---
- name: Install and start Nginx
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes
    - name: Start Nginx Service
      service:
        name: nginx
        state: started
        enabled: yes
```

Explanation:

1. Define a play
2. Target hosts (group: webservers)
3. Become root (sudo)
4. Install Nginx using apt module
5. Start and enable Nginx service

AD-HOC COMMANDS EXAMPLES

Command	Description
`ansible all -m ping`	Check connectivity to all hosts
`ansible web -m command -a "uptime"`	Run uptime command on web group
`ansible db -m shell -a "df -h"`	Run shell command on db group
`ansible all -m yum -a "name=httpd state=present"`	Install httpd package on all hosts
`ansible all -m service -a "name=httpd state=started"`	Start httpd service on all hosts
`ansible all -m copy -a "src=/tmp/a.txt dest=/opt/"`	Copy file to remote host

INVENTORY EXAMPLE

```
[webservers]
web1  ansible_host=192.168.1.10
web2  ansible_host=192.168.1.11

[dbservers]
db1   ansible_host=192.168.1.20
db2   ansible_host=192.168.1.21

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

[webservers:vars]
http_port=80
```

ANSIBLE MODULE CATEGORIES

- **System:** user, group, service, package, reboot
- **Files:** copy, file, template, fetch
- **Commands:** command, shell, script
- **Network:** network_cli, netconf, ios_config
- **Cloud:** ec2, s3, rds, azure_rm
- **Others:** cron, firewalld, selinux, hostname

COMMONLY USED MODULES

Module	Purpose
ping	Check connectivity
command / shell	Run commands on remote host
yum / apt	Install packages
service / systemd	Manage services
copy	Copy files to remote host
file	Create/remove files or directories
template	Use Jinja2 templates
user / group	Manage users and groups
lineinfile	Modify lines in a file
cron	Manage cron jobs

BEST PRACTICES

- Keep playbooks simple and modular
- Use meaningful names for tasks
- Use variables instead of hardcoding values
- Store sensitive data in Ansible Vault
- Test playbooks in lower environment first
- Use roles for reusability
- Keep inventory organized
- Use version control (Git) for playbooks

QUICK TIPS

- `ansible-playbook site.yml` → Run a playbook
- `ansible-inventory --list` → List inventory
- `ansible all -m setup` → Gather facts
- `ansible-vault create secret.yml` → Encrypt file
- `ansible-doc <module_name>` → Get module docs

INSTALL ANSIBLE (QUICK)

```
# On Ubuntu
sudo apt update
sudo apt install ansible -y

# On RHEL/CentOS
sudo yum install epel-release -y
sudo yum install ansible -y
```

Ansible makes infrastructure management SIMPLE, POWERFUL and AUTOMATED!

10. CLOUD + DEVOPS

Deliver Faster. Scale Smarter. Operate Better.

WHAT IS CLOUD?

Cloud Computing is the delivery of computing services (servers, storage, databases, networking, software, analytics, etc.) over the internet (on-demand) with pay-as-you-go pricing.

WHAT IS DEVOPS?

DevOps is a culture and set of practices that bring Development (Dev) and Operations (Ops) team together to build, test, release and monitor software faster and more reliably.



CLOUD + DEVOPS = POWER COMBO

- ✓ On-demand scalable infrastructure
- ✓ Faster provisioning & deployments
- ✓ High availability & resiliency
- ✓ Cost optimization
- ✓ Global reach
- ✓ Focus on innovation, not infrastructure

CLOUD SERVICE MODELS

IaaS
(Infrastructure as a Service)

You manage OS, middleware, runtime, data & applications. Provider manages infrastructure.

Examples: AWS EC2, Azure VM, Google Compute Engine

PaaS
(Platform as a Service)

You manage application & data. Provider manages runtime, middleware, OS & infrastructure.

Examples: AWS Elastic Beanstalk, Azure App Service, Google App Engine

SaaS
(Software as a Service)

Complete software delivered over the internet. You just use it.

Examples: Gmail, GitHub, Salesforce, Office 365

CLOUD DEPLOYMENT MODELS

Public Cloud
Owned by provider. Shared infrastructure. (Examples: AWS, Azure, GCP)

Private Cloud
Dedicated to one organization. (On-Prem / Hosted)

Hybrid Cloud
Combination of Public + Private Cloud.

Multi-Cloud
Use of multiple cloud providers.

HOW CLOUD ENHANCES DEVOPS

- ✓ Elastic infrastructure for dynamic workloads
- ✓ Faster environments (Dev/Test/Prod) on demand
- ✓ Global availability & disaster recovery
- ✓ Managed services reduce operational overhead
- ✓ Integrates well with CI/CD pipelines
- ✓ Monitoring, logging & security at scale
- ✓ Cost savings with auto-scaling & pay-as-you-go

CI/CD PIPELINE IN CLOUD

Code	Build	Test	Package	Deploy	Monitor
Developers commit code to repo	Compile code & create build artifacts	Run automated tests (unit, integration, security, etc.)	Package the application (Docker image, etc.)	Deploy to cloud infrastructure (Dev/Staging/Prod)	Monitor, log & get alerts. Continuously improve

Feedback Loop (Monitor → Learn → Improve)

POPULAR CLOUD PROVIDERS

Amazon Web Services
- Largest market share
- 200+ services
- Strong in compute, storage, databases, serverless

Microsoft Azure
- Great for enterprises
- Deep integration with Microsoft ecosystem
- Hybrid cloud strong

Google Cloud
- Strong in data, analytics & AI/ML
- Kubernetes (GKE) leader
- Global network fast

KEY CLOUD SERVICES FOR DEVOPS

- **Compute** (EC2, VM, Compute Engine) — Run your applications
- **Storage** (S3, Blob, GCS) — Store code, artifacts, backups
- **Database** (RDS, Cloud SQL, Cosmos DB) — Managed databases
- **Networking** (VPC, Subnet, Load Balancer) — Secure & scalable connectivity
- **CI/CD** (CodePipeline, DevOps, Cloud Build) — Managed pipeline services
- **Container Services** (ECS, AKS, GKE) — Run & orchestrate containers
- **Serverless** (Lambda, Functions, Cloud Run) — Run code without managing servers
- **Monitoring** (CloudWatch, Azure Monitor, Stackdriver) — Monitor, alert & auto-heal

INFRASTRUCTURE AS CODE (IaC) IN CLOUD

Provision & manage cloud resources using code.

Terraform
(HashiCorp)

- ✓ Consistent
- ✓ Repeatable
- ✓ Versioned
- ✓ Auto-scaling & self-healing

Example: Terraform (AWS EC2)

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  key_name      = "my-key"
  tags = {
    Name = "WebServer"
  }
}
```

CONTAINERIZATION & ORCHESTRATION IN CLOUD

Docker
Package app & dependencies in containers

Kubernetes
Orchestrate containers (auto-scaling, self-healing, rolling updates)

Cloud Managed K8s
- EKS (AWS)
- AKS (Azure)
- GKE (GCP)

CLOUD SECURITY IN DEVOPS

- ✓ Identity & Access Management (IAM)
- ✓ Encrypt data in transit & at rest
- ✓ Security Groups / Firewall rules
- ✓ Secrets Management (Vault, KMS)
- ✓ Regular vulnerability scanning
- ✓ Compliance & audit logging

BEST PRACTICES

- ✓ Automate everything (IaC, CI/CD)
- ✓ Use least privilege access
- ✓ Monitor everything, set alerts
- ✓ Tag resources & manage costs
- ✓ Use multi-region for high availability
- ✓ Backup, test DR regularly
- ✓ Continuously optimize & improve

USE CASES

- Web Application Hosting
- Microservices Deployment
- CI/CD Pipelines
- Data Analytics & AI/ML
- Disaster Recovery & Backup
- Global Content Delivery

CLOUD + DEVOPS JOURNEY

1. Learn Basics Cloud & Linux
2. Automation (IaC, Scripts)
3. CI/CD Pipelines
4. Containers & K8s
5. Monitoring & Observability
6. Optimize & Scale

KEY TAKEAWAY

Cloud provides the foundation. DevOps provides the speed. Together they deliver value faster, better & at scale!

CLOUD GIVES YOU POWER. DEVOPS GIVES YOU SPEED. TOGETHER YOU WIN!

11. CI/CD PIPELINE DESIGN

Automate Build. Test Thoroughly. Deploy Confidently. Deliver Continuously.

WHAT IS CI/CD?

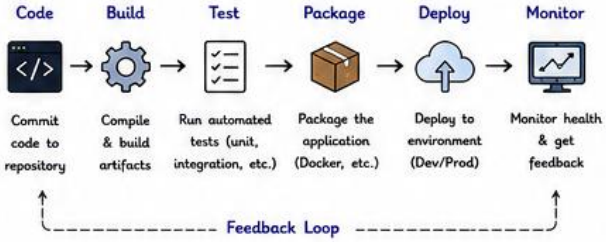
CI/CD is the practice of automating the integration (CI) of code changes and automating the delivery (CD) of applications to production.

- **CI (Continuous Integration):** Frequently integrate code changes into a shared repository and validate them.
- **CD (Continuous Delivery/Deployment):** Automatically deliver validated code to staging/production environments.

GOALS

- Deliver features faster
- Improve code quality
- Reduce manual errors
- Increase deployment frequency
- Enable quick rollback
- Ensure consistent & reliable releases

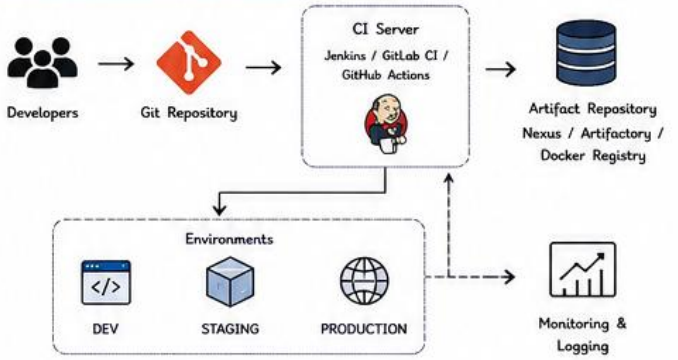
CI/CD PIPELINE OVERVIEW



TYPICAL CI/CD PIPELINE STAGES

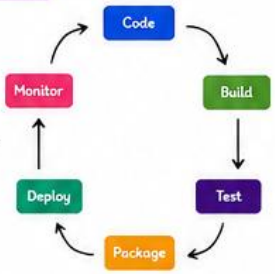
Stage	Purpose	Example Tools
1. Code (Commit)	Developers write code and push changes to version control system.	Git, GitHub, GitLab, Bitbucket
2. Build	Fetch code, resolve dependencies, compile/build the code.	Maven, Gradle, npm, Docker
3. Test	Run automated tests to ensure code quality.	JUnit, PyTest, Jest, Selenium, SonarQube
4. Code Quality	Static code analysis, linting, security scanning.	SonarQube, ESLint, Checkmarx, Snyk
5. Package	Package the application into an artifact or container image.	Docker, Nexus, Artifactory, npm, JAR, WAR
6. Deploy to Staging	Deploy to staging environment for validation.	Kubernetes, Docker, Helm, Ansible
7. Integration / E2E Test	Run integration and end-to-end tests on staging.	Selenium, Cypress, Postman, Karate
8. Approve / Gate	Manual approval or quality gate before production.	Human approval, Quality Gates
9. Deploy to Production	Deploy the approved build to production.	Kubernetes, AWS, Azure, Ansible, Helm
10. Monitor & Notify	Monitor application health and notify teams.	Prometheus, Grafana, ELK, Slack, PagerDuty

PIPELINE ARCHITECTURE (HIGH LEVEL)



EXAMPLE: PIPELINE FLOW (WEB APPLICATION)

1. Developer pushes code to GitHub.
2. CI server pulls code.
3. Build stage compiles code & runs unit tests.
4. Code quality tools scan for issues.
5. Package stage builds Docker image and stores in registry.
6. Deploy to Staging environment.
7. Run integration & E2E tests.
8. If all tests pass, deploy to Production.
9. Monitor application and send alerts if any issue.



EXAMPLE PIPELINE: GITLAB CI (YAML)

```yaml
stages:
  - build
  - test
  - package
  - deploy

build:
  stage: build
  script:
    - npm install
    - npm run build

test:
  stage: test
  script:
    - npm run test

package:
  stage: package
  script:
    - docker build -t myapp:$CI_COMMIT_SHA .
    - docker push registry.example.com/myapp:$CI_COMMIT_SHA

deploy:
  stage: deploy
  script:
    - kubectl set image deployment/myapp myapp=registry.example.com/myapp:$CI_COMMIT_SHA
  environment:
    name: production
  only:
    - main
```

What this does?

- Builds the application
- Runs tests
- Builds & pushes Docker image
- Deploys to Kubernetes
- only on main branch

PIPELINE BEST PRACTICES

- Keep pipeline fast and fail early.
- Automate as much as possible.
- Run tests in parallel to save time.
- Use caching to speed up builds.
- Keep configurations in code (IaC).
- Secure secrets with vaults/variables.
- Use containers for consistent builds.
- Monitor pipeline health and success rate.
- Implement rollback strategy.
- Continuously improve pipeline.

PIPELINE DESIGN CONSIDERATIONS

- Choose the right tools for your tech stack.
- Design for security (secrets, access, scans).
- Use environments that mirror production.
- Implement approvals for critical deployments.
- Enable blue-green or canary deployments.
- Ensure rollback and disaster recovery.
- Optimize for cost and performance.

TOOLS ECOSYSTEM

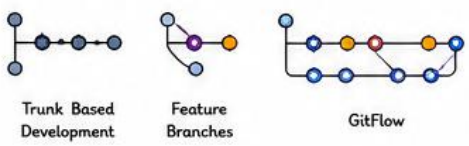
Source Control	CI/CD	Build & Package	Artifact Repo	Deploy & Orchestrate	Monitoring & Logging	Notifications
GitHub, GitLab, Bitbucket	Jenkins, GitLab CI, GitHub Actions	Maven, Gradle, Docker	Nexus, Artifactory, Docker Hub	Kubernetes, Helm, Ansible	Prometheus, Grafana, ELK Stack	Slack, PagerDuty, Email

TYPES OF CD

Continuous Delivery
Code is always kept in a deployable state. Deployment to production is manual.

Continuous Deployment
Every change that passes the pipeline is automatically deployed to production.

COMMON PIPELINE PATTERNS



KEY TAKEAWAY

A well-designed CI/CD pipeline helps you deliver high-quality software faster, with fewer errors and higher confidence!

AUTOMATE EVERYTHING YOU CAN. VALIDATE EVERYTHING YOU DEPLOY. DELIVER VALUE CONTINUOUSLY.

12. MONITORING + LOGGING

Observe. Detect. Understand. Act.



You can't improve what you don't monitor.

WHAT IS MONITORING?

Monitoring is the process of collecting and analyzing real-time data from systems, applications and infrastructure to ensure performance, availability and reliability.

- Track health & performance
- Detect issues early
- Alert teams
- Improve user experience



WHAT IS LOGGING?

Logging is the process of collecting, storing and analyzing log data generated by applications, servers and services.

- Understand what happened
- Troubleshoot issues
- Audit & compliance
- Root cause analysis



WHY IT MATTERS?

- High availability & reliability
- Faster issue detection & resolution
- Proactive performance optimization
- Better security & compliance
- Data-driven decision making



TYPES OF MONITORING

Infrastructure Monitoring	Application Monitoring	Service Monitoring	Database Monitoring	User Experience Monitoring
CPU, Memory, Disk, Network, Load, etc.	Response time, Throughput, Error rate, Apdex, etc.	Availability of services, APIs, Endpoints	Query performance, Connections, Locks, Slow queries	Real user monitoring, Page load time, User flows

KEY METRICS (GOLDEN SIGNALS)

Latency	Time taken to serve a request (e.g., API response time)
Traffic	Number of requests / transactions per second/minute
Errors	Rate of failed requests (e.g., 4xx, 5xx errors)
Saturation	Utilization of resources (CPU, Memory, Disk, Network)

USE CASES

- Real-time issue detection
- Alerting on critical events
- Performance bottlenecks
- Capacity planning
- Security monitoring
- Business insights

MONITORING & LOGGING ARCHITECTURE (HIGH LEVEL)

SOURCES
- Applications
- Servers
- Databases
- Containers
- Network Devices
- Cloud Services

→

COLLECTORS / AGENTS
- Metric Collectors (Prometheus Agent, Node Exporter)
- Log Collectors (Filebeat, Fluentd, Vector)
- APM Agents (OpenTelemetry, Jaeger, New Relic)

→

DATA PIPELINE
- Metrics Pipeline (Prometheus, InfluxDB)
- Logs Pipeline (Elastic, Loki)
- Traces Pipeline (Tempo, Jaeger)

→

STORAGE
- Time Series DB (Prometheus, InfluxDB)
- Log Storage (Elasticsearch, S3)
- Trace Storage (Tempo, Jaeger)

→

VISUALIZATION & ALERTING
- Dashboards (Grafana, Kibana)
- Alerting (Alertmanager, PagerDuty)
- Notifications (Email, Slack, SMS)

→

USERS
- DevOps Team
- Developers
- SRE / Ops
- Management

POPULAR MONITORING TOOLS

Prometheus	Metrics collection & alerting
Grafana	Visualization & dashboards
Datadog	Infrastructure & APM monitoring
New Relic	Full-stack observability
Zabbix	Infrastructure monitoring

POPULAR LOGGING TOOLS

- **ELK Stack (Elasticsearch, Logstash, Kibana)** — Centralized logging, search & visualization
- **Loki** — Log aggregation system by Grafana Labs
- **Graylog** — Log management & analysis
- **Splunk** — Log management, search & analytics
- **Amazon CloudWatch Logs** — Managed log service by AWS

APM / TRACING TOOLS

- **OpenTelemetry** — Standard for collecting telemetry data
- **Jaeger** — Distributed tracing system
- **Tempo** — High volume trace storage
- **Zipkin** — Distributed tracing

ALERTING BEST PRACTICES

- Alert on symptoms, not causes
- Avoid alert fatigue
- Use meaningful thresholds
- Group similar alerts
- Provide runbook / next steps
- Route alerts to the right team
- Review and tune alerts regularly



LOGGING BEST PRACTICES

- Log at the right level (INFO, WARN, ERROR)
- Use structured logging (JSON)
- Include timestamps & correlation IDs
- Avoid logging sensitive data
- Centralize logs
- Set log retention & rotation policies



DASHBOARD BEST PRACTICES

- Keep it simple and focused
- Use meaningful visualizations
- Show trends, not just numbers
- Add annotations & context
- Drill-down capability
- Refresh at the right interval



COMMON ALERT SEVERITIES

P1	Critical	Immediate action
P2	High	High priority
P3	Medium	Needs attention
P4	Low	Informational

EXAMPLE QUERIES

PromQL (Prometheus)

- High CPU Usage

```
100 - (avg by (instance)(irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

- HTTP 5xx Error Rate

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m])) * 100
```

Loki Query

- Error logs from last 1 hour

```
{level="error"} |= "exception" | line_format "{{.time}} {{.message}}"
```

- Logs from a specific service

```
{app="checkout-service"} | json | line_format "{{.level}} - {{.msg}}"
```

MONITORING & LOGGING CHECKLIST

- All critical systems are monitored
- Alerts are configured & tested
- Dashboards are created & shared
- Logs are centralized & searchable
- Retention policies are defined
- Teams are notified & on-call ready





GOOD OBSERVABILITY = HAPPY USERS + HAPPY TEAMS + RELIABLE SYSTEMS



13. SECURITY + DEVSECOPS



Build Secure. Ship Secure. Run Secure. Stay Secure.

WHAT IS SECURITY?

Security is the practice of protecting systems, applications and data from unauthorized access, attacks and damage.

- Confidentiality
- Integrity
- Availability
- Authentication
- Authorization
- Non-repudiation

WHAT IS DEVSECOPS?

DevSecOps integrates security into every stage of the DevOps lifecycle. Security is not a phase, it's a shared responsibility.

- Shift security left
- Automate security
- Continuously monitor
- Respond and improve



WHY DEVSECOPS?

- Catch vulnerabilities early
- Reduce security risks & breaches
- Faster & secure releases
- Compliance & audit readiness
- Build trust with users
- Lower cost of security fixes

DEVSECOPS LIFECYCLE (SHIFT LEFT SECURITY)

Plan	Code	Build	Test	Release	Deploy	Operate	Monitor
• Threat Modeling • Security Requirements	• Secure Coding • Pre-commit Hooks	• SAST • Dependency Scan • Secrets Scan	• DAST • IAST • Pen Testing	• Artifact Scan • Image Scan • Policy Check	• IaC Scan • KBs Security Checks	• Runtime Protection • Access Control	• SIEM Logging • Alerts & Response

Continuous Feedback

SECURITY PRINCIPLES

- **Least Privilege** — Give only necessary access.
- **Defense in Depth** — Use multiple layers of security.
- **Fail Secure** — Default should be secure.
- **Secure by Default** — Make security the default choice.
- **Continuous Verification** — Never trust, always verify.

COMMON THREATS

- Injection (SQLi, Command Injection)
- Broken Authentication
- Sensitive Data Exposure
- Broken Access Control
- Security Misconfiguration
- Vulnerable Dependencies
- Cross Site Scripting (XSS)
- Insecure Deserialization

SECURITY IN CI/CD PIPELINE

Code	Build	Test	Package	Deploy	Monitor
SAST Secrets Scan Code Review	Dependency Scan SCA	DAST IaST Unit Test	Container Scan Artifact Scan	Policy Check IaC Scan KBs Scan	RASP SIEM Alerts

Automate • Integrate • Monitor • Respond

SECURITY TOOLS ECOSYSTEM

SAST	SonarQube, Checkmarx, Semgrep, CodeQL
SCA / Dependency Scan	Snyk, OWASP Dependency Check, GitHub Dependabot
Secrets Management	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
Container Security	Trivy, Aqua Trivy, Anchore Grype, Clair
DAST	OWASP ZAP, Burp Suite, Nuclei
IaC Security	Checkov, tfsec, Terrascan, Kics
Runtime Security	Falco, Sysdig Secure, Aqua, Tracee
SIEM / Monitoring	Splunk, ELK Stack, Datadog, Microsoft Sentinel

SECURE CODING BEST PRACTICES

- Validate all inputs
- Use parameterized queries
- Sanitize output to prevent XSS
- Use HTTPS everywhere
- Implement proper authentication and authorization
- Do not hardcode secrets
- Handle errors securely
- Keep dependencies up to date

SECRETS MANAGEMENT

- Never hardcode secrets
- Use environment variables
- Use Secrets Manager / Vault
- Rotate secrets regularly
- Restrict access (Least Privilege)
- Audit secret usage

POLICY AS CODE (EXAMPLE - OPA / REGOPA)

```
package kubernetes.admission

deny[msg] {
    input.request.kind.kind == "Pod"
    not input.request.object.spec.containers[_].securityContext.runAsNonRoot
    msg = "Containers must run as non-root user"
}

deny[msg] {
    input.request.kind.kind == "Pod"
    input.request.object.spec.containers[_].image
    not startswith(input.request.object.spec.containers[_].image, "myregistry.com/")
    msg = "Use approved container images"
}
```

OPA

DEVSECOPS BEST PRACTICES

- Integrate security early (Shift Left)
- Automate security in pipelines
- Use policy as code
- Keep dependencies & base images updated
- Continuously monitor & log
- Perform regular security training
- Collaborate: Dev, Sec, Ops as one team

COMPLIANCE & STANDARDS

- OWASP Top 10
- ISO 27001
- SOC 2
- PCI DSS
- GDPR
- HIPAA
- CIS Benchmarks

THREAT MODELING

- Identify assets
- Identify threats
- Analyze vulnerabilities
- Determine impact
- Mitigate risks
- Review & iterate

INCIDENT RESPONSE (HIGH LEVEL)

1	Detect	Identify the incident
2	Contain	Limit the impact
3	Eradicate	Remove the cause
4	Recover	Restore systems
5	Lessons Learned	Improve & prevent

KEY TAKEAWAYS

- Security is everyone's responsibility.
- Automate security, don't rely on manual checks.
- Continuously monitor, detect and respond.
- Measure, improve and stay compliant.

SECURE TODAY, STRONGER TOMORROW!

DEV + SEC + OPS = TRUST + SPEED + RESILIENCE

14. DEPLOYMENT STRATEGIES + RELEASE MANAGEMENT

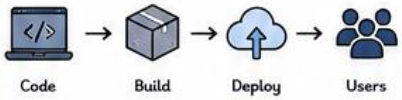
Plan → Build → Test → Deploy → Monitor → Improve

DEV OPS

Right Strategy. Right Time. Right Delivery.

WHAT IS DEPLOYMENT?

Deployment is the process of making a software application available for use in a production environment.



WHAT IS RELEASE MANAGEMENT?

Release Management is the process of planning, scheduling, coordinating and controlling a software release through all stages and environments.

- Ensure high quality releases
- Minimize risk and downtime
- Improve communication & visibility
- Enable fast and reliable delivery

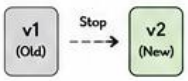
GOALS

- Deliver value faster
- Reduce deployment failures
- Enable quick rollback
- Ensure stability & reliability
- Improve customer satisfaction
- Maintain compliance & traceability

COMMON DEPLOYMENT STRATEGIES

1. Recreate / Big Bang

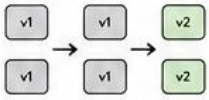
Stop old version and deploy new version.



- Simple
- Downtime, high risk

2. Rolling Deployment

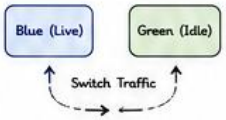
Deploy new version to a subset of servers at a time.



- Minimal downtime
- Slower process

3. Blue/Green Deployment

Maintain two identical environments.



- Zero downtime
- Double infrastructure cost

4. Canary Deployment

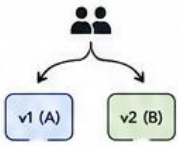
Release to a small group of users before full rollout.



- Low risk
- Complex monitoring

5. A/B Testing

Run two versions and let users choose.



- Data driven decisions
- Needs more traffic

6. Feature Toggle

Deploy feature in code but keep it OFF. Enable via toggle.



- Safe & flexible
- Code complexity

RELEASE MANAGEMENT LIFECYCLE



Plan	Build	Test	Release	Deploy	Verify	Close
Define scope, timeline, resources and risks.	Develop and integrate changes.	Functional, non-functional and regression tests.	Package, version and prepare release notes.	Deploy to target environment using strategy.	Validate, monitor and collect feedback.	Document, review and close the release.

TYPES OF RELEASES

- **Major Release:** Significant new features or major changes.
- **Minor Release:** Improvements, minor new features.
- **Patch Release:** Bug fixes and small updates.
- **Hotfix Release:** Urgent fixes for critical issues.
- **Rollout Release:** Gradual release to users/regions.

DEPLOYMENT STRATEGY COMPARISON

Strategy	Downtime	Risk	Speed	Infra Cost	Use Case
Recreate / Big Bang	High	High	Fast	Low	Small apps, non-critical
Rolling Deployment	Low	Medium	Medium	Medium	Most applications
Blue/Green Deployment	None	Low	Fast	High	Mission critical apps
Canary Deployment	Very Low	Very Low	Medium	Medium	Customer facing apps
A/B Testing	None	Low	Slow	High	Feature validation
Feature Toggle	None	Very Low	Fast	Low	Frequent releases

RELEASE MANAGEMENT BEST PRACTICES

- Plan releases and communicate early.
- Follow semantic versioning.
- Automate as much as possible.
- Maintain release checklist.
- Use deployment strategies to reduce risk.
- Test in production-like environments.
- Monitor, validate and rollback if needed.
- Document everything and run retrospectives.

ROLLBACK STRATEGIES

- **Instant Rollback (Rerreate)** — Stop new version and redeploy previous stable version.
- **Blue/Green Switch Back** — Switch traffic back to previous stable environment.
- **Rolling Rollback** — Roll back instances to previous version in batches.
- **Feature Toggle Off** — Turn off the feature via toggle without redeploying.

WHEN TO ROLLBACK?

- High error rate
- Performance degradation
- Failed health checks
- User complaints
- Business impact
- Data inconsistency

RELEASE MANAGEMENT TOOLS

Jenkins	GitLab CI/CD	Argo CD	Azure DevOps	Octopus Deploy	Harness	AWS Code Deploy
Automation Server	CI/CD & GitOps	GitOps CD Tool	CI/CD & Release	Release Automation	Continuous Delivery	Deployment Service

RELEASE CHECKLIST (EXAMPLE)

- Code merged and reviewed
- Build successful
- All tests passed
- Security scan passed
- Release notes updated
- Stakeholders notified
- Backup / Rollback plan ready
- Monitoring and alerts in place
- Approval received
- Deployed successfully
- Post deployment validation done
- Release closed and documented

KEY TAKEAWAY

A good deployment strategy reduces risk.
A strong release management process ensures consistency.
Together, they deliver software faster, safer and better.

PLAN WELL • TEST THOROUGHLY • DEPLOY SMART • IMPROVE CONTINUOUSLY

FAST DELIVERY WITHOUT BREAKING THINGS = HAPPY USERS + HAPPY TEAMS

15. DEVOPS TOOLS + QUICK REVISION NOTES

Collaborate · Automate · Integrate · Deploy · Monitor · Improve

DEVOPS TOOLS ECOSYSTEM

Plan	Code	Build	Test	Release	Deploy	Operate	Monitor
Project Mgmt. Jira Trello Asana	**SCM** Git GitHub GitLab Bitbucket	**Build Tools** Maven Gradle NPM Yarn	**Testing** Selenium JUnit TestNG Postman	**CI Servers** Jenkins GitLab CI CircleCI Travis CI	**Containers** Docker Podman	**Config Mgmt.** Ansible Chef Puppet	**Monitoring** Prometheus Grafana Datadog Nagios
Docs / Wiki Confluence Notion	**Code Review** GitHub PR GitLab MR	**Artifacts** Nexus JFrog Artifactory	**Quality** SonarQube Checkstyle	**CD Tools** Argo CD Spinnaker	**Orchestration** Kubernetes OpenShift **IaC / Provisioning** Terraform AWS CloudFormation	**Secrets Mgmt.** HashiCorp Vault	**Logging** ELK Stack Loki

DEVOPS QUICK REVISION NOTES

1. **DevOps = Development (Dev) + Operations (Ops)**
   Culture, practices and tools that help deliver software faster, more reliably and with continuous improvement.
2. **Core Principles**
   Collaboration, Automation, Continuous Integration, Continuous Delivery, Monitoring, Feedback & Learning.
3. **DevOps Lifecycle**
   Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Improve (Repeat)
4. **CI vs CD**
   CI: Frequently integrate code and run automated tests.
   CD: Automatically deliver code to production.
5. **Infrastructure as Code (IaC)**
   Manage infrastructure using code.
   Tools: Terraform, CloudFormation, Ansible.
6. **Containers**
   Package application + dependencies to run anywhere.
   Tools: Docker, Podman.
7. **Orchestration**
   Automate deployment, scaling, networking, self-healing.
   Tool: Kubernetes.
8. **Configuration Management**
   Manage system configuration and dependencies.
   Tools: Ansible, Chef, Puppet.
9. **Monitoring & Logging**
   Monitor health & performance. Logs for troubleshooting.
   Tools: Prometheus, Grafana, ELK Stack.
10. **Security / DevSecOps**
    Security in every stage. Shift-left security.
    Tools: Snyk, Trivy, Vault, SonarQube.

IMPORTANT CONCEPTS (ONE-LINERS)

- **Git:** Distributed Version Control System.
- **Branch:** Isolated line of development.
- **Pull Request / Merge Request:** Code review before merge.
- **Pipeline:** Automated workflow of steps.
- **Artifact:** Output of build (JAR, WAR, Docker image).
- **Container:** Lightweight, portable runtime unit.
- **Image:** Read-only template used to create containers.
- **Kubernetes:** Orchestrates containers at scale.
- **Pod:** Smallest deployable unit in K8s (one or more containers).
- **Service:** Stable endpoint to access pods.
- **Ingress:** Manage external access to services.
- **IaC:** Provision & manage infra through code.
- **Configuration Management:** Maintain desired state.
- **Secret Management:** Store secrets securely.
- **Monitoring:** Metrics & alerts for system health.
- **Logging:** Centralized logs for analysis.

DEVOPS BEST PRACTICES

- Automate everything you can.
- Version everything (code, infra, configs).
- Keep builds small, fast and repeatable.
- Test early and test often.
- Use Infrastructure as Code.
- Monitor everything, alert on time.
- Keep environments as similar as possible.
- Secure by default, not as an afterthought.
- Document and share knowledge.
- Continuously measure and improve.



GIT COMMANDS (CHEAT SHEET)

`git clone <repo-url>`	Clone a repository
`git status`	Show current status
`git add .`	Stage all changes
`git commit -m "msg"`	Commit changes
`git push origin <branch>`	Push to remote
`git pull origin <branch>`	Pull from remote
`git branch`	List branches
`git checkout <branch>`	Switch branch
`git merge <branch>`	Merge branch
`git log --oneline --graph`	Show commit history

Commit small, push often, pull before push.

DOCKER COMMANDS (BASICS)

`docker build -t myapp .`	Build image
`docker images`	List images
`docker run -d -p 8080:80 myapp`	Run container
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <id>`	Stop container
`docker rm <id>`	Remove container
`docker rmi <image>`	Remove image

Build once, run anywhere.

KUBECTL COMMANDS (BASICS)

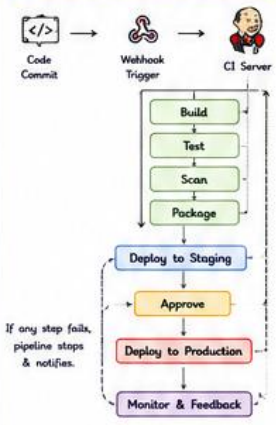
`kubectl get pods`	List pods
`kubectl get svc`	List services
`kubectl get deployments`	List deployments
`kubectl describe <resource>`	Details of resource
`kubectl logs <pod>`	View logs
`kubectl apply -f file.yaml`	Apply configuration
`kubectl delete -f file.yaml`	Delete configuration
`kubectl rollout status deploy/<name>`	Check rollout status

Declarative > Imperative.

COMMON PIPELINE STAGES

Checkout	Fetch code from SCM
Build	Compile / Build the code
Test	Run automated tests
Scan	Security / Code quality scan
Package	Create artifact / Image
Deploy	Deploy to target environment
Verify	Smoke / Integration tests
Notify	Send notifications / Alerts

PIPELINE EXAMPLE (HIGH LEVEL)



TROUBLESHOOTING CHEAT SHEET

- **Build Failures** — Check logs, dependencies, config.
- **Tests Failing** — Check test cases, environment.
- **Deployment Failing** — Check configs, permissions, resources.
- **Pod Not Starting** — Check events: kubectl describe pod
- **High CPU / Memory** — Scale, optimize, or increase resources.
- **No Logs** — Check log path, sidecar, permissions.

Check logs first, then configs.

DEVOPS MINDSET

- **Automate** — Remove manual steps.
- **Collaborate** — Break silos, share ownership.
- **Measure** — Use metrics for decisions.
- **Learn** — Continuously learn & adapt.
- **Deliver Value** — Focus on customer value.

"We don't deploy code, We deliver value."

ONE PAGE RECAP

- **Culture** — People, process, collaboration
- **Automation** — CI/CD, IaC, Config Mgmt.
- **Delivery** — Fast, reliable, repeatable
- **Operate** — Monitor, log, alert, respond
- **Security** — DevSecOps, shift-left
- **Tools** — Use right tool for the job
- **Goal** — Happy users, happy teams



REMEMBER ★ — PLAN WELL → CODE SMART → BUILD FAST → TEST THOROUGHLY → DEPLOY CONFIDENTLY → MONITOR CONTINUOUSLY → IMPROVE ALWAYS → **THAT'S DEVOPS!** ★